

MLDL Experiment 10

Aim: Explore and implement Autoencoders for image denoising.

Theory:

1. Dataset Source

The dataset used for this experiment is the **Fashion MNIST** dataset, provided by Zalando Research.

- **Source Link:** [Keras Datasets - Fashion MNIST](#) / [Kaggle Dataset Link](#)

2. Dataset Description

Fashion MNIST was designed as a direct "drop-in" replacement for the original MNIST digit dataset.

- **Format:** 28 * 28 grayscale images.
- **Size:** 70,000 images total (60,000 training, 10,000 testing).
- **Features:** 784 pixels (flattened) or a 28 \times 28 \times 1 tensor. Each pixel value ranges from 0 to 255.
- **Classes:** 10 categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot).
- **Characteristics:** Unlike digits, these images contain more complex textures and edges, making them an ideal benchmark for denoising tasks.

3. Mathematical Formulation

An Autoencoder is a neural network designed to learn a compressed, distributed representation (encoding) of a dataset. For denoising, we use a **Denoising Autoencoder (DAE)**.

The Mapping Process

1. **Encoder (f_θ):** Maps the noisy input $x + \delta$ to a latent representation (bottleneck) h .

$$h = f_\theta(x + \delta)$$

2. **Decoder (g_ϕ):** Maps the latent representation h back to the reconstructed image $\hat{x}$.

$$\hat{x} = g_\phi(h)$$

The Objective Function

The goal is to minimize the reconstruction error between the **clean** original image x and the **reconstructed** image $\hat{x}$. Since we treat pixel intensities as probabilities, we use **Binary Cross-Entropy (BCE)**:

$$\mathcal{L}(x, \hat{x}) = - \sum_i [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$$

By minimizing this loss, the model learns to capture the "signal" and ignore the "noise" δ .

4. Algorithm Limitations

- **Blurriness:** Autoencoders tend to produce blurry outputs because the MSE/BCE loss functions encourage the model to "play it safe" by averaging pixel values.
- **Structured Noise:** DAEs work best with random Gaussian noise. If the noise has a specific structure (like a line drawn across an image), the model might mistake it for a feature.
- **Information Bottleneck:** If the latent space (bottleneck) is too small, the model will lose critical details. If it is too large, the model might simply "memorize" the noise.
- **Computational Cost:** Convolutional layers with many filters require significant GPU memory and time for high-resolution images.

5. Methodology / Workflow

The workflow follows a standard deep-learning pipeline:

1. **Data Loading:** Fetching the 2D arrays from Keras.
2. **Preprocessing:** Normalization to $[0, 1]$ and reshaping to 4D tensors (Batch, H, W, C).
3. **Noise Injection:** Adding Gaussian noise with a **noise_factor** of 0.5 and clipping values to maintain the valid range.
4. **Architecture Design:**
 - **Encoder:** Two **Conv2D** layers for feature extraction followed by **MaxPooling2D** to reduce spatial dimensions.
 - **Decoder:** Two **Conv2D** layers followed by **UpSampling2D** to restore the image to 28×28 .
5. **Training:** Fitting the model using **adam** optimizer and **binary_crossentropy** loss.
6. **Evaluation:** Visualizing the "Noisy Input" vs. "Denoised Output."

6. Performance Analysis

Based on your results:

- **Convergence:** The loss decreased from ~ 0.33 to ~ 0.29 within 10 epochs, suggesting the model learned effectively.
- **Visual Fidelity:** The "Denoised Output" successfully removed the grain. While there is a slight loss in sharpness (typical for AEs), the objects (like the coat and trousers) are clearly recognizable.
- **Metric Interpretation:** A `val_loss` of **0.2948** indicates that the model has reached a stable state where the reconstruction error is minimized relative to the noise level.

7. Hyperparameter Tuning

To optimize the results, the following parameters were/can be adjusted:

- **Noise Factor:** Reduced from 0.5 to 0.3 if the noise is too aggressive for the model to recover edges.
- **Number of Filters:** Increasing filters (e.g., from 32 to 64) allows the model to learn more complex textures.
- **Latent Space Size:** Adjusting the depth of the bottleneck layer determines how much "compression" occurs.
- **Optimizer Learning Rate:** Tuning the Adam learning rate (e.g., 0.001 to 0.0005) can lead to a smoother loss curve.
- **Impact:** Increasing epochs and filter count generally leads to sharper reconstructions but increases training time.

8. Code

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model

# 1. Load & Preprocess Data
# We don't need the labels (y_train/test) for an autoencoder
(x_train, _), (x_test, _) = fashion_mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape to (28, 28, 1) for the Convolutional layers
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
```

```

x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# 2. Add Synthetic Noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)

# Ensure pixel values stay within the valid [0, 1] range
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# 3. Build the Convolutional Autoencoder
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)

# At this point the representation is (7, 7, 32)

# Decoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Define and Compile
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# 4. Train the Model
# We train by mapping: Noisy Image -> Original Clean Image
autoencoder.fit(x_train_noisy, x_train,
                epochs=10,
                batch_size=128,
                shuffle=True,
                validation_data=(x_test_noisy, x_test))

# 5. Test (Predict)

```

```

decoded_imgs = autoencoder.predict(x_test_noisy)

# 6. Display Results
n = 10
plt.figure(figsize=(20, 4))
for i in range(1, n + 1):
    # Display Noisy Input
    ax = plt.subplot(2, n, i)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 1: ax.set_title("Noisy Input")

    # Display Denoised Output
    ax = plt.subplot(2, n, i + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 1: ax.set_title("Denoised Output")

plt.show()

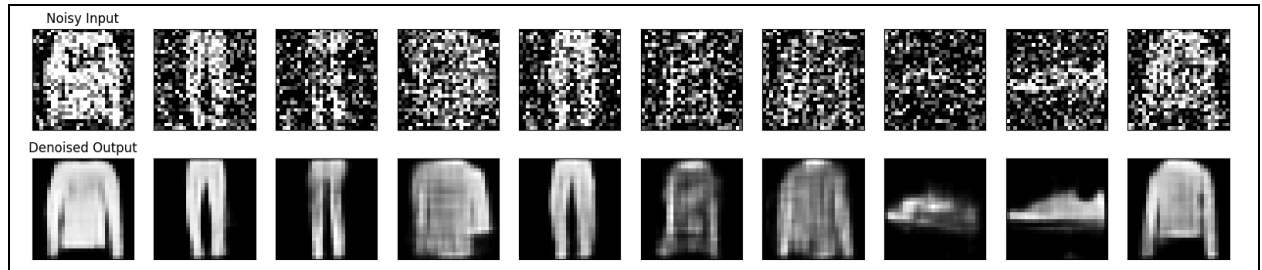
```

9. Output

```

Epoch 1/10
469/469 ————— 11s 11ms/step - loss: 0.3325 - val_loss: 0.3111
Epoch 2/10
469/469 ————— 3s 6ms/step - loss: 0.3054 - val_loss: 0.3053
Epoch 3/10
469/469 ————— 3s 6ms/step - loss: 0.3016 - val_loss: 0.3022
Epoch 4/10
469/469 ————— 3s 6ms/step - loss: 0.2992 - val_loss: 0.3004
Epoch 5/10
469/469 ————— 3s 6ms/step - loss: 0.2975 - val_loss: 0.2987
Epoch 6/10
469/469 ————— 3s 6ms/step - loss: 0.2960 - val_loss: 0.2972
Epoch 7/10
469/469 ————— 3s 6ms/step - loss: 0.2949 - val_loss: 0.2968
Epoch 8/10
469/469 ————— 3s 6ms/step - loss: 0.2940 - val_loss: 0.2962
Epoch 9/10
469/469 ————— 3s 6ms/step - loss: 0.2933 - val_loss: 0.2952
Epoch 10/10
469/469 ————— 3s 6ms/step - loss: 0.2926 - val_loss: 0.2948
313/313 ————— 1s 3ms/step

```



Conclusion

The experiment successfully demonstrates the capability of **Convolutional Autoencoders** to perform non-linear dimensionality reduction for image denoising. By forcing the data through a bottleneck, the model learns to prioritize global structural patterns over local random variations. For the Fashion MNIST dataset, the DAE effectively recovers the shape and category of clothing items even when the input is heavily corrupted by Gaussian noise.